Reproducibility of Published Results: Type Soundness of the Simple MIRA Language

Eduardo Sandalo Porto

Department of Computer Science, Federal University of Minas Gerais.

1 Introduction

This report discusses the results of reproducing the artifact of the paper *A Framework for End-to-End Verification and Evaluation of Register Allocators* [2], presented at the Fourteenth International Static Analysis Symposium (SAS 2007).

The paper presents a framework for designing, verifying, and evaluating register allocation algorithms. Specifically, it presents the MIRA and FORD languages, which, respectively, describe programs prior to register allocation, and describe the results produced by the register allocation. It also discusses RALF, which allows register allocator integration with GCC. To verify the correctness of MIRA and FORD, the paper defines the Simple MIRA (sMIRA) and Simple FORD (sFORD) languages, defines an operational semantics and a type system for FORD, and states the soundness theorems for this type system with respect to the semantics.

The proofs for these theorems are available in an artifact, written in Twelf [3]. The original artifact can be found at `http://compilers.cs.ucla.edu/ralf/twelf/`. The goal of this report is to evaluate these proofs, and translate them to the Lean 4 proof assistant [1]. The translated proofs are available in a `git` repository at `https://github.com/edusporto/smira-lean`.

2 Artifact

I was able to download each of the individual proof files from the website available at the previously mentioned URL. I was not able to download the `zip` file that aggregated the rest, so there could be information missing. I managed to build the Twelf prover from the instructions found at `https://twelf.org/download/`. Adding a `sources.cfg` file listing the artifact files (`arith.elf`, `list.elf`, `os_smira.elf`, `tp_smira.elf`, `sub.thm`, `smira.thm`), I was able to successfully check the proofs using `twelf-server` and running `make sources.cfg`. Each file contains the following:

- `arith.elf`: Definition for natural numbers.
- `list.elf`: Definition for linked lists (`exps`) over the expressions (`exp`).
- `os_smira.elf`: A small-step operational semantics for the languages.
- `tp_smira.elf`: A type system for the languages.
- `sub.thm`: Proofs about the subtyping relation of the type system.
- `smira.thm`: The main proofs of *progress* and *preservation* for the type system, which together imply soundness.

Note that the definitions and theorems are both about the sMIRA and the sFORD languages, although the file names only include sMIRA. The type system's goal is to guarantee that the values used in the sMIRA program are preserved in the sFORD representation. Thus, the syntax allows *pseudo-registers*, which are only available in sMIRA, but the typing rule for the `mov` instruction forces the left operand to be a *physical register*, which is a characteristic of sFORD. There are a few components described in the paper missing, such as the `call`, `load`, `store` operators and the data heap Δ. Nonetheless, the definitions and proofs seem to be correct for the subset they represent.

3 Reproducibility

The goal of this effort was to translate the proofs to Lean 4, keeping proofs and definitions similar to the original Twelf code while also utilizing a few of Lean's capabilities. We divide the translation into five parts: the definition of the abstract-syntax tree (AST), the operational semantics, the typing rules, the subtyping relation rules, and the final soundness proofs. Before that, we mention a detail which comes from the Twelf proofs. Although Lean includes natural numbers and linked lists whose definitions are similar to Twelf's, we added three inductive predicates to make the translation of proofs easier: `NotZero`, `Proj`, `Update`. There are other ways to encode these relations in Lean, but changing them would diverge the structure of Lean's and Twelf's proofs.

```
/-- Ensures a natural number is strictly greater than zero.
    (Twelf's `not_zero` relation). -/
inductive NotZero : Nat → Prop where [...]

/-- List projection.
    `Proj xs n x` means the `n`-th element of list `xs` is `x`.
    (Twelf's `proj_exp` relation). -/
inductive Proj {α : Type} : List α → Nat → α → Prop where [...]

/-- List update.
    `Update xs x' n xs' x` means that the resulting of updating the `n`-th element
    in `xs` yields `xs'`, where `x` is the old value.
    (Twelf's `update_exp` relation). -/
inductive Update {α : Type} : List α → α → Nat → List α → α → Prop where [...]
```

Since lists in Lean are polymorphic, we don't need to define specific lists of expressions as in Twelf.

3.1 AST

The AST of sMIRA contains operands, labels, instructions, sequences of instructions, and a code heap (program). Its metatheory includes register banks and states.

The operands are pseudo-registers, which represent variables; bullets, which don't affect the allocation process; and physical registers.

```
inductive Operand where
  /-- Pseudo-register (variable). -/
  | psd : Nat → Operand
  /-- Bullet, which has no effect in the allocation process. -/
  | blt : Operand
  /-- Pre-colored register.
      Represents a pair `(r_n, p_n)`, where `r_n` is a position in the register bank,
      and `p_n` is the pseudo stored in that position. -/
  | reg : Nat → Nat → Operand
```

We define sequences of instructions as *basic blocks*, although that term is imprecise in this context. We do it to group lists of instructions with an unconditional jump, representing the end of the block. This is the only departure from the Twelf code, which represented jumps as a normal instruction. The other instructions represented are moves and conditional jumps. A label is simply an index in an instruction list. The program (code heap) is a list of basic blocks.

```
/-- Label of a basic block.
    A program is a list of blocks, whose label is its index in the instruction list. -/
structure Label where
  id : Nat

/-- Program instruction (i). -/
inductive Inst where
  | mov : Operand → Operand → Inst
  | cnd : Operand → Label → Inst

/-- A basic block of instructions (I). -/
inductive BasicBlock where
  | cons : Inst → BasicBlock → BasicBlock
  | jump : Label → BasicBlock

/-- The code heap (C), or the program.
    It is essentially a lookup table where index `j` represents the label `j`. -/
abbrev Program := List BasicBlock
```

Finally, the register bank represents the allocation of registers, and the state of a program is the current basic block with the current register bank.

```
/-- Register bank (R). -/
abbrev RegisterBank := List Operand

/-- The mutable execution state: the current basic block (I) and the register bank (R). -/
abbrev State := BasicBlock × RegisterBank
```

3.2 Operational semantics

Based on the evaluation of operands, we define the small-step operational semantics representing the state transitions triggered by the execution of each instruction:

```
inductive Eval : RegisterBank → Operand → Operand → Prop where
  /-- Evaluating a bullet just yields a bullet. -/
  | blt : ∀ {R}, Eval R Operand.blt Operand.blt
  /-- Evaluating a physical register `(reg r_n p_n)` yields the pseudo `(psd p_n)`, provided that projecting
    index `r_n` out of the bank `R` yields `(psd p_n)`, and the pseudo ID `p_n` is strictly greater than zero
    (i.e., it is not in the uninitialized state). -/
  | reg : ∀ {R r_n p_n},
      Proj R r_n (Operand.psd p_n) →
      -- `p_n = 0` represents the non-initialized registers ⊥
      NotZero p_n →
      Eval R (Operand.reg r_n p_n) (Operand.psd p_n)

/-- Small-step operational semantics for a given program C. -/
inductive Step (C : Program) : State → State → Prop where
  | mov : ∀ {R r_n p_n o I R' v oOld},
      Eval R o v →
      Update R (Operand.psd p_n) r_n R' oOld →
      Step C
        (BasicBlock.cons (Inst.mov (Operand.reg r_n p_n) o) I, R)
        (I, R')
  | jmp : ∀ {R lbl I'},
      Proj C lbl.id I' →
      Step C
        (BasicBlock.jump lbl, R)
        (I', R)
  | jeq : ∀ {R r_n p_n lbl I v},
      Eval R (Operand.reg r_n p_n) v →
      Step C
        (BasicBlock.cons (Inst.cnd (Operand.reg r_n p_n) lbl) I, R)
        (I, R)
  | jne : ∀ {R r_n p_n lbl I I' v},
      Eval R (Operand.reg r_n p_n) v →
      Proj C lbl.id I' →
      Step C
        (BasicBlock.cons (Inst.cnd (Operand.reg r_n p_n) lbl) I, R)
        (I', R)
```

Lean allows us to easily define user-provided notation. We define the following notation for small-step operational semantics, stepping from state s_1 to s_2 given program C:

```
-- Small-step notation: C ⊢ s₁ ⇒ s₂
notation C " ⊢ " s₁ " ⇒ " s₂ => Step C s₁ s₂
```

3.3 Type system

First, we define the possible types of operands in the register allocator. We use them to define the type environment Γ, which represents the shape of the register bank, then use Γ to define the program type, which maps labels to their expected entry environments.

```
/-- Types in the register allocator. -/
inductive Ty where
  /-- Type of the bullet `blt` operand. -/
  | const : Ty
  /-- Type of a register holding a pseudo `p_n`. -/
  | psd : Nat → Ty

/-- Register type environment (Γ). -/
abbrev TypeEnv := List Ty

/-- Program (code heap) type (Ψ). -/
abbrev TypeHeap := List TypeEnv
```

The type system depends on the notion of *environment subtyping*. We say that $\Gamma \sqsubseteq \Gamma'$ if and only if every live pseudo-variable required by the target environment Γ' is exactly matched by the source environment Γ, while registers considered dead by Γ' place no restrictions on Γ. This is useful for the typing of branching instructions, showing that the target environment of a jump will be valid according to the original environment.

```
/-- `Sub Γ Γ'` means environment `Γ` is a valid subtype of `Γ'`. -/
inductive Sub : TypeEnv → TypeEnv → Prop where
  | empbk : Sub [] []
  /-- If the target block expects `psd 0` (uninitialized/dead), the current block can safely provide
      any pseudo `p_n`. -/
  | rzero : ∀ {n Γ Γ'},
      Sub Γ Γ' →
      Sub (Ty.psd n :: Γ) (Ty.psd 0 :: Γ')
  /-- If the target expects a live pseudo (succ n), we must provide that live pseudo. -/
  | rnotz : ∀ {n Γ Γ'},
      Sub Γ Γ' →
      Sub (Ty.psd (Nat.succ n) :: Γ) (Ty.psd (Nat.succ n) :: Γ')
```

We then define the typing rules (or *judgements*) that construct the types of the system. To avoid extending the report, we merely show the typing rules without going into further detail. We define custom syntax for each typing rule to closely resemble the paper. Note that we use subscripts to differentiate the judgements—Lean is also able to define ambiguous notation, which would be solved via type inference.

```
/-- Type judgement of operands. -/
inductive TyOp : TypeEnv → Operand → Ty → Prop where
  | gab : ∀ {Γ}, TyOp Γ Operand.blt Ty.const
  | reg : ∀ {Γ r_n p_n},
      Proj Γ r_n (Ty.psd p_n) →
      NotZero p_n →
      TyOp Γ (Operand.reg r_n p_n) (Ty.psd p_n)
notation:50 Γ " ⊢_p " o " : " τ => TyOp Γ o τ

/-- Type judgement of instructions. -/
inductive TyInst : TypeHeap → TypeEnv → Inst → TypeEnv → Prop where
  | mov : ∀ {Ψ Γ_0 Γ_1 r_n p_n o τ pOld},
      (Γ_0 ⊢_p o : τ) →
      NotZero p_n →
      Update Γ_0 (Ty.psd p_n) r_n Γ_1 pOld →
      TyInst Ψ Γ_0 (Inst.mov (Operand.reg r_n p_n) o) Γ_1
  | cnd : ∀ {Ψ Γ Γ' r_n p_n lbl T_o},
      (Γ ⊢_p (Operand.reg r_n p_n) : T_o) →
      Proj Ψ lbl.id Γ' →
      (Γ ⊑ Γ') →
      TyInst Ψ Γ (Inst.cnd (Operand.reg r_n p_n) lbl) Γ
notation:50 Ψ " ⊢_i " inst " : " Γ_0 " ↦ " Γ_1 => TyInst Ψ Γ_0 inst Γ_1

/-- Type judgement of basic blocks. -/
inductive TyBlock : TypeHeap → TypeEnv → BasicBlock → Prop where
```

```
  | cons : ∀ {Ψ Γ Γ' i I},
      (Ψ ⊢ᵢ i : Γ ↦ Γ') →
      TyBlock Ψ Γ' I →
      TyBlock Ψ Γ (BasicBlock.cons i I)
  | jump : ∀ {Ψ Γ Γ' lbl},
      Proj Ψ lbl.id Γ' →
      (Γ ⊑ Γ') →
      TyBlock Ψ Γ (BasicBlock.jump lbl)
notation:50 Ψ " ⊢ₛ " I " : " Γ => TyBlock Ψ Γ I

/-- Type judgement of the register bank. -/
inductive TyRegBank : RegisterBank → TypeEnv → Prop where
  | nil: TyRegBank [] []
  | cons : ∀ {pₙ R' Γ'},
      TyRegBank R' Γ' →
      TyRegBank (Operand.psd pₙ :: R') (Ty.psd pₙ :: Γ')
notation:50 "⊢ᵣ " R " : " Γ => TyRegBank R Γ

/-- Auxiliary lemma for the code heap type judgement.
    This is kept for closer similarity to the Twelf code - there are other ways of doing this in Lean. -/
inductive TyCodeHeapAux : TypeHeap → Program → TypeHeap → Prop where
  | nil : ∀ {Ψ}, TyCodeHeapAux Ψ [] []
  | cons : ∀ {Ψ C Γ Ψ'} {I : BasicBlock},
      (Ψ ⊢ₛ I : Γ) →
      TyCodeHeapAux Ψ C Ψ' →
      TyCodeHeapAux Ψ (I :: C) (Γ :: Ψ')

/-- Type judgement of the program (code heap). -/
inductive TyCodeHeap : TypeHeap → Program → Prop where
  | mk : ∀ {Ψ C},
      TyCodeHeapAux Ψ C Ψ →
      TyCodeHeap Ψ C
notation:50 "⊢ₕ " C " : " Ψ => TyCodeHeap Ψ C
```

Finally, the type judgement of the machine state, which is the subject of the progress and preservation rules, is the following:

```
/-- Type judgement of the machine state. -/
inductive TyState : Program → State → Prop where
  | mk : ∀ {Ψ Γ Γ'} {C : Program} {R : RegisterBank} {I : BasicBlock},
      (⊢ₕ C : Ψ) →
      (⊢ᵣ R : Γ) →
      (Ψ ⊢ₛ I : Γ') →
      (Γ ⊑ Γ') →
      TyState C (I, R)
notation:50 "⊢ₘ " "⟨" C ", " s "⟩" => TyState C s
```

3.4 Proofs

In our translation, we define and prove an equivalent statement for each theorem stated in the Twelf files. To avoid extending the report further, we show only the statements for the progress and preservation proofs. Notably, proofs in Lean tend to be slightly longer than proofs in Twelf. Although dependent elimination in Lean prunes pattern matching cases in many of the theorems, Twelf's totality checker and terser syntax leads to slightly smaller proofs.

The *progress* theorem states that if we are in a machine state in program `C` with basic block `I` and register bank `R`, then there must always exist another state `s'` such that the current state can step into it. Note that there is no termination in the language.

```
theorem progress {C : Program} {s : State}
    (hTy : ⊢ₘ ⟨C, s⟩) :
    ∃ s', C ⊢ s ⇒ s'
```

The *preservation* theorem states that if we are in a valid machine state $\langle C, s_1 \rangle$ and we know that we can step into a state s_2, then the machine state $\langle C, s_2 \rangle$ is also valid.

```
theorem preservation {C : Program} {s₁ s₂ : State}
    (hTy : ⊢ₘ ⟨C, s₁⟩)
    (hStep : C ⊢ s₁ ⇒ s₂) :
    ⊢ₘ ⟨C, s₂⟩
```

4 Conclusion

In this report, we briefly showed the main parts of translated definitions in Lean 4 from Twelf. Although downloading the full Twelf code and running the proofs was quite easy, I found it very difficult to understand the Twelf definitions and proofs. Twelf's theory is based on LF, while Lean (which I have more experience in) is based on the Calculus of Constructions. Thus, Twelf code takes a logical programming approach, while Lean gravitates towards classical functional programming. One of the hardest parts of Twelf for me is internalizing the inputs and outputs of each proof, which I still find confusing even considering the explicit definition of in and out parameters. I also think that the addition of custom syntax available in Lean makes it easier to understand definitions and proofs. Since the notation used is very similar to the paper, while writing the proofs I was able to look at the paper and more easily understand what was being proved at each point.

Unlike Twelf, our Lean formalization based on inductive propositions does not automatically yield executable semantics or an executable type checker. To achieve executability, we would have to implement computable functions for these and prove their equivalence to the rules we wrote. However, because Lean is a functional language, implementing these functions is straightforward and arguably more extensible than the Twelf code. Furthermore, by utilizing Lean's extensible syntax, we could define custom notation for the AST to fully parse and run the sMIRA language as an embedded domain-specific language (eDSL) in Lean.

For instance, we could write these executable functions and formally prove their equivalence to our inductive theorems, guaranteeing that the runnable eDSL adheres to the sMIRA specification:

```
def step (C : Program) (s : State) : Option State := [...]

theorem step_equiv {C : Program} {s1 s2 : State} :
    (step C s1 = some s2) ↔ (C ⊢ s1 ⇒ s2) := [...]

def checkState (C : Program) (s : State) : Bool := [...]

theorem checkState_equiv {C : Program} {s : State} :
    checkState C s = true ↔ ⊢ₘ ⟨C, s⟩ := [...]
```

Finally, note that the proofs written in this project are similar to the ones in Twelf. Lean's automation capabilities could reduce much of the proof code, but we want to stay close to the original representation to improve direct comparison between the languages.

Acknowledgements

The paper whose artifact was reproduced in this report was presented in a class by Prof. Fernando Magno Quintão Pereira, who is one of its authors. I would like to thank Prof. Fernando for the classes on the subject, and the classes on Twelf. I would also like to disclose the usage of LLMs to help understand some of the Twelf code, as well as to refine some of the Lean definitions and proofs.

References

[1] Leonardo de Moura and Sebastian Ullrich. "The Lean 4 Theorem Prover and Programming Language". en. In: *Automated Deduction – CADE 28*. Ed. by André Platzer and Geoff Sutcliffe. Cham: Springer International Publishing, 2021, pp. 625–635. ISBN: 978-3-030-79876-5. DOI: 10.1007/978-3-030-79876-5_37.

[2] V. Krishna Nandivada, Fernando Magno Quintão Pereira, and Jens Palsberg. "A Framework for End-to-End Verification and Evaluation of Register Allocators". en. In: *Static Analysis*. Ed. by Hanne Riis Nielson and Gilberto Filé. Berlin, Heidelberg: Springer, 2007, pp. 153–169. ISBN: 978-3-540-74061-2. DOI: 10.1007/978-3-540-74061-2_10.

[3] Frank Pfenning and Carsten Schürmann. "System Description: Twelf — A Meta-Logical Framework for Deductive Systems". en. In: *Automated Deduction — CADE-16*. Ed. by Harald Ganzinger. Berlin, Heidelberg: Springer, 1999, pp. 202–206. ISBN: 978-3-540-48660-2. DOI: 10.1007/3-540-48660-7_14.